

✓ Colab'a hoş geldiniz.

Google Colab artık VS Code'da kullanılabilir.



Visual Studio Code için yeni [Google Colab uzantısını](#) deneyin. Birkaç tıklamayla kullanmaya başlayabilirsiniz:

- VS Code'da **Uzantılar** görünümünü açın ve "Google Colab"i arayarak uzantıyı yükleyin.
- Yerel çalışma alanınızda herhangi bir `.ipynb` not defteri dosyası oluşturun veya açın ve bir hücreyi çalıştırarak ya da sağ üstteki **Çekirdek Seç** düğmesini tıklayarak çekirdek seçiciyi açın.
- **Colab**'i tıklayın, istediğiniz çalışma zamanını seçin ve Google Hesabınızla oturum açın. Artık hazırsınız.

Daha fazla ayrıntı için [duyurunun yapıldığı blogumuzu buradan](#) inceleyin.

🎁 ABD'deki Üniversite Öğrencilerine Gemini ve Colab Pro Planı Ücretsiz 🎓

Colab'ın, veri bilimi ve makine öğrenimine yönelik özel yüksek işlem kaynaklarıyla desteklenen en doğru modelimiz Gemini 3 Pro'ya daha fazla erişim elde ederek gelişmiş kodlar, karmaşık araştırmalar ve yenilikçi projeler oluşturun.

Ücretsiz Gemini teklifini [gemini.google/students](#) adresinden kullanabilirsiniz.

Ücretsiz Colab teklifini [colab.research.google.com/signup](#) adresinden kullanabilirsiniz.

Şartlar Geçerlidir.

✓ API anahtarı olmadan, Google-Colab-AI aracılığıyla popüler yapay zeka modellerine erişin

Tüm kullanıcılar, `google-colab-ai` Python kitaplığı aracılığıyla en popüler LLM'lere, ücretli planı olan kullanıcılar ise daha geniş bir model yelpazesine erişebilir. Daha fazla bilgi için [Google Colab AI'yi kullanmaya başlama](#) başlıklı makaleyi inceleyin.

```
from google.colab import ai
response = ai.generate_text("What is the capital of France?")
```

✓ Gemini API'yi keşfedin

Gemini API, Google DeepMind tarafından oluşturulan Gemini modellerine erişmenizi sağlar. Gemini modelleri baştan sona çok formatlı olacak şekilde tasarlandığından metin, görsel, kod ve seslerle ilgili sorunsuz şekilde mantık yürütebilirsiniz.

Kullanmaya başlama adımları

- [Google AI Studio](#)'ya gidip Google Hesabınızla giriş yapın.
- [API anahtarı oluşturma](#).
- [Python](#) hızlı başlangıç kılavuzunu kullanın veya [curl](#) kullanarak REST API'yi çağırın.

Gemini'nin gelişmiş özelliklerini keşfedin

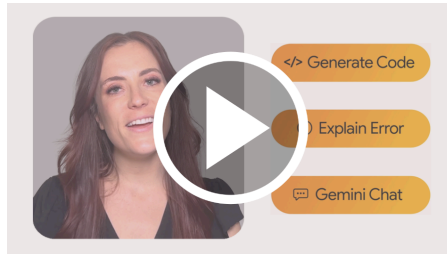
- Metin ile görüntüleri tekrar tekrar karıştırarak Gemini'nin [çok formatlı çıktılarıyla](#) denemeler yapın.
- [Çok formatlı Live API](#)'yi keşfedin (demoyu [burada](#) bulabilirsiniz).
- Gemini'yi kullanarak [görüntüleri analiz etmeyi ve resimlerinizdeki öğeleri tespit etmeyi](#) öğrenin (ayrıca [3D versiyonlar](#) da mevcuttur).
- Karmaşık görevleri kendi düşünceleriyle çözebilen [Gemini düşünme modelinin](#) gücünden faydalanın.

Karmaşık kullanım alanlarını keşfedin

- Modelin internette bulabileceği bilgilere dayalı olarak bir şirket hakkında rapor oluşturmak için [Gemini temellendirme özelliklerini](#) kullanın.
- [PDF'lerdeki fatura ve form verilerini](#) yapılandırılmış bir şekilde ayıklayın.
- Gemini'nin büyük bağlam penceresini ve Imagen'i kullanarak [bir kitabın tamamına dayalı resimler](#) oluşturun.

Daha fazla bilgi edinmek için [Gemini çözüm kitabını](#) veya [Gemini API belgelerini](#) inceleyin.

Colab'de artık [Gemini](#) tarafından desteklenen yapay zeka özellikleri sunuluyor. Python'u yeni kullanmaya başlayanlar veya deneyimli kullanıcılar için aşağıdaki videoda bu özelliklerin nasıl kullanılacağı açıklanmaktadır.



Colab nedir?

Colab veya diğer adıyla "Colaboratory", tarayıcınızda Python'u yazmanızı ve çalıştırmanızı sağlar.

- Hiç yapılandırma gerektirmez
- GPU'lara ücretsiz erişim
- Kolay paylaşım imkanı sunar

İster **öğrenci** ister **veri bilimci** ister **yapay zeka araştırmacısı** olun, Colab işinizi kolaylaştırabilir. Daha fazla bilgi edinmek için [Introduction to Colab](#) (Colab'e Giriş) veya [Colab Features You May Have Missed](#) (Kaçırmış Olabileceğiniz Colab Özellikleri) videolarını izleyebilir ya da aşağıdan hemen kullanmaya başlayabilirsiniz.

▼ Başlarken

Okuduğunuz doküman statik bir web sayfası değil, kod yazmanıza ve yürütmenize imkan veren **Colab not defteri** adında etkileşimli bir ortamdır.

Örneğin, buradaki **kod hücresinde**, bir değeri hesaplayan, bir değişken içinde saklayan ve sonucu yazdıran kısa bir Python dizesi görebilirsiniz:

```
seconds_in_a_day = 24 * 60 * 60
seconds_in_a_day

86400
```

Yukarıdaki hücrede kodu yürütmek için tıklayarak seçin, ardından ya kodun sol tarafındaki oynat düğmesine basın ya da "Command/Ctrl+Enter" klavye kısayolunu kullanın. Kodu düzenlemek için hücreyi tıklamanız yeterlidir. Sonrasında düzenlemeye başlayabilirsiniz.

Bir hücrede tanımladığınız değişkenler daha sonra başka hücrelerde kullanılabilir:

```
seconds_in_a_week = 7 * seconds_in_a_day
seconds_in_a_week

604800
```

Colab not defterleri; **yürütülebilir kod**, **zengin metin**, **resimler**, **HTML**, **LaTeX** ve diğer öğeleri tek bir dokümanda birleştirmenizi sağlar. Oluşturduğunuz Colab not defterleri Google Drive hesabınızda saklanır. Colab not defterlerinizi arkadaşlarınızla veya iş arkadaşlarınızla kolayca paylaşabilir, not defterlerinize yorum yapmalarını, hatta düzenlemelerini sağlayabilirsiniz. Daha fazla bilgiyi [Colab'e Genel Bakış](#) bölümünde bulabilirsiniz. Yeni bir Colab not defteri oluşturmak için yukarıdaki Dosya menüsünü ya da [yeni bir Colab not defteri oluşturma](#) bağlantısını kullanabilirsiniz.

Colab not defterleri, Colab tarafından barındırılan Jupyter not defterleridir. Jupyter projesi hakkında daha fazla bilgiyi jupyter.org adresinde bulabilirsiniz.

▼ Veri bilimi

Colab ile popüler Python kitaplıklarının tüm avantajlarından yararlanarak veri analiz edip görselleştirebilirsiniz. Aşağıdaki kod hücresi rastgele veri oluşturmak için **numpy**'yi, bu veriyi görselleştirmek için de **matplotlib**'i kullanır. Kodu düzenlemek için hücreyi tıklamanız yeterlidir. Sonrasında düzenlemeye başlayabilirsiniz.

Kendi verilerinizi Google Drive hesabınızdan (e-tablolar dahil), GitHub'dan ve diğer pek çok kaynaktan Colab not defterlerine aktarabilirsiniz. Veri içe aktarma ve Colab'ın veri bilimi için nasıl kullanılabileceği hakkında daha fazla bilgi edinmek için [Verilerle Çalışma](#) bölümünün altındaki bağlantılara bakabilirsiniz.

```
import numpy as np
import IPython.display as display
```

```

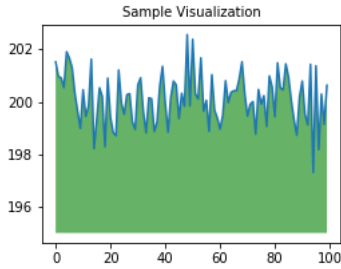
from matplotlib import pyplot as plt
import io
import base64

ys = 200 + np.random.randn(100)
x = [x for x in range(len(ys))]

fig = plt.figure(figsize=(4, 3), facecolor='w')
plt.plot(x, ys, '-')
plt.fill_between(x, ys, 195, where=(ys > 195), facecolor='g', alpha=0.6)
plt.title("Sample Visualization", fontsize=10)

data = io.BytesIO()
plt.savefig(data)
image = F"data:image/png;base64,{base64.b64encode(data.getvalue()).decode()}"
alt = "Sample Visualization"
display.display(display.Markdown(F"!!![{alt}]({image})"))
plt.close(fig)

```



Colab not defterleri Google'ın bulut sunucularında kod yürütür. Yani makinenizin gücünden bağımsız olarak, [GPU'lar ve TPU'lar](#) dahil Google donanımının gücünden yararlanabilirsiniz. Tek ihtiyacınız olan şey bir tarayıcıdır.

Örneğin, **pandas** kodunun çalışmasını tamamlamasını bekliyor ve daha hızlı şekilde ilerlemek istiyorsanız bir GPU Çalışma Zamanı'na geçebilirsiniz ve kod değişikliği gerektirmeden hızlanma sağlayan [RAPIDS cuDF](#) gibi kitaplıkları kullanabilirsiniz.

Colab'da pandas'ı hızlandırma hakkında daha fazla bilgi edinmek için [10 dakikalık kılavuzu](#) veya [ABD menkul kıymetler piyasası veri analizi demosunu](#) inceleyin.

✓ Makine öğrenimi

Colab ile bir resim veri kümesini içe aktarabilir, üzerinde bir resim sınıflandırıcıyı eğitebilir ve modeli değerlendirebilirsiniz. Hem de sadece [birkaç satır kodla](#).

Colab, makine öğrenimi topluluğunda yaygın olarak şu uygulamalarla kullanılır:

- TensorFlow'u kullanmaya başlama
- Nöral ağ geliştirme ve eğitme
- TPU'lar ile deneme yapma
- Yapay zeka araştırmalarını yayma
- Eğitici oluşturma

Makine öğrenimi uygulamalarını açıklayarak tanıtan örnek Colab not defterlerini görmek için aşağıdaki [makine öğrenimi örneklerine](#) bakabilirsiniz.

✓ Diğer Kaynaklar

Colab'de Not Defterleriyle Çalışma

- [Colab'e genel bakış](#)
- [Markdown rehberi](#)
- [Kitaplıkları içe aktarma ve bağımlıları yükleme](#)
- [GitHub'da not defteri kaydetme ve yükleme](#)
- [Etkileşimli formlar](#)
- [Etkileşimli widget'lar](#)

Verilerle Çalışma

- [Veri yükleme: Drive, E-Tablolar ve Google Cloud Storage](#)

- [Grafikler: Veri görselleştirme](#)
- [BigQuery'yi kullanmaya başlama](#)

Makine Öğrenimi

Google'ın online Makine Öğrenimi kursu da dahil olmak üzere Makine Öğrenimi ile ilgili birkaç not defterini burada bulabilirsiniz. Daha fazlası için [tam kurs web sitesine](#) bakın.

- [Pandas DataFrame'e giriş](#)
- [Pandas'ı hızlandırmak için RAPIDS cuDF'ye giriş](#)
- [cuML'nin hızlandırıcı modunu kullanmaya başlama](#)

Hızlandırılmış Donanım Kullanma

- [Flax NNX API kullanarak MNIST veri kümesindeki el yazısıyla yazılmış rakamları sınıflandırmak için bir CNN eğitin](#)
- [JAX ile görüntü sınıflandırma için bir Vision Transformer \(ViT\) eğitin](#)
- [JAX kullanarak dönüştürücü dil modeliyle metin sınıflandırma](#)

▼ Öne çıkan örnekler

- [JAX AI Stack ile bir miniGPT dil modelini eğitin](#)
- [Tunix kullanarak LLM için LoRA/QLoRA ince ayarı](#)
- [LoRA ve QLoRA ile Gemma'nın parametre açısından verimli ince ayarı](#)
- [Hugging Face dönüştürücüleri kontrol noktalarını yükleme](#)
- [Keras'ta 8 bitlik tam sayı nicemleme](#)
- [Basit bir dönüştürücü modeliyle Float8 eğitimi ve çıkarımı](#)
- [KerasHub ile dönüştürücüye sıfırdan ön eğitim verme](#)
- [Basit MNIST evrişimli sinir ağı](#)
- [Keras 3 kullanarak sıfırdan görüntü sınıflandırma](#)
- [KerasHub ile görüntü sınıflandırma](#)

```
import torch
import torch.nn as nn
import torch.optim as optim
import matplotlib.pyplot as plt
from PIL import Image
from torchvision import models, transforms
import torch.nn.functional as F

# GPU kontrolü
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

# 1. Boş bir AlexNet modeli kuruyoruz
model_alexnet = models.alexnet(weights=None)

# 2. Son sınıflandırıcı katmanını 2 sınıfa uyarlıyoruz
num_fts = model_alexnet.classifier[6].in_features
model_alexnet.classifier[6] = nn.Linear(num_fts, 2)

# 3. Modeli ekran kartına taşıyoruz
model_alexnet = model_alexnet.to(device)

# Senin orijinal kodundaki gibi kriter ve optimizasyonu tanımlıyoruz
kriter = nn.CrossEntropyLoss()
optimizasyon = optim.Adam(model_alexnet.parameters(), lr=0.001)

print("AlexNet modeli ve optimizasyon başarıyla hazırlandı!")
```

AlexNet modeli ve optimizasyon başarıyla hazırlandı!

```
print("Eğitim başlıyor... Lütfen bekleyin...")
print("-" * 40)

# Senin gönderdiğin orijinal otomatik eğitim döngüsü
for epoch in range(15):
    model_alexnet.train()
    running_loss = 0.0
    running_corrects = 0

    for inputs, labels in veri_yukleyiciler['train']:
        inputs, labels = inputs.to(device), labels.to(device)
        optimizasyon.zero_grad()

        outputs = model_alexnet(inputs)
```

```

_, preds = torch.max(outputs, 1)
loss = kriter(outputs, labels)

loss.backward()
optimizasyon.step()

running_loss += loss.item() * inputs.size(0)
running_corrects += torch.sum(preds == labels.data)

epoch_loss = running_loss / len(görüntü_veri_setleri['train'])
epoch_acc = running_corrects.double() / len(görüntü_veri_setleri['train'])

model_alexnet.eval()
val_loss = 0.0
val_corrects = 0
with torch.no_grad():
    for inputs, labels in veri_yukleyiciler['val']:
        inputs, labels = inputs.to(device), labels.to(device)
        outputs = model_alexnet(inputs)
        _, preds = torch.max(outputs, 1)
        loss = kriter(outputs, labels)

        val_loss += loss.item() * inputs.size(0)
        val_corrects += torch.sum(preds == labels.data)

val_epoch_loss = val_loss / len(görüntü_veri_setleri['val'])
val_epoch_acc = val_corrects.double() / len(görüntü_veri_setleri['val'])

# Bilgisayarın tamamen kendi hesapladığı yüzdeler ekrana basılıyor
print(f"Epoch [{epoch+1}/15]")
print(f"  Eğitim    -> Kayıp: {epoch_loss:.4f} | Doğruluk: %{epoch_acc*100:.2f}")
print(f"  Doğrulama -> Kayıp: {val_epoch_loss:.4f} | Doğruluk: %{val_epoch_acc*100:.2f}")
print("-" * 40)

print("Eğitim Bitti! Model tamamen otomatik olarak eğitildi.")

```

```

Doğrulama -> Kayıp: 0.6927 | Doğruluk: %54.00
-----
Epoch [2/15]
Eğitim    -> Kayıp: 0.7009 | Doğruluk: %50.00
Doğrulama -> Kayıp: 0.6918 | Doğruluk: %54.00
-----
Epoch [3/15]
Eğitim    -> Kayıp: 0.6937 | Doğruluk: %51.00
Doğrulama -> Kayıp: 0.6938 | Doğruluk: %46.00
-----
Epoch [4/15]
Eğitim    -> Kayıp: 0.6944 | Doğruluk: %47.00
Doğrulama -> Kayıp: 0.6992 | Doğruluk: %46.00
-----
Epoch [5/15]
Eğitim    -> Kayıp: 0.7042 | Doğruluk: %50.00
Doğrulama -> Kayıp: 0.6960 | Doğruluk: %46.00
-----
Epoch [6/15]
Eğitim    -> Kayıp: 0.6941 | Doğruluk: %50.00
Doğrulama -> Kayıp: 0.6938 | Doğruluk: %46.00
-----
Epoch [7/15]
Eğitim    -> Kayıp: 0.6932 | Doğruluk: %50.00
Doğrulama -> Kayıp: 0.6937 | Doğruluk: %46.00
-----
Epoch [8/15]
Eğitim    -> Kayıp: 0.6932 | Doğruluk: %51.00
Doğrulama -> Kayıp: 0.6926 | Doğruluk: %54.00
-----
Epoch [9/15]
Eğitim    -> Kayıp: 0.6939 | Doğruluk: %46.00
Doğrulama -> Kayıp: 0.6929 | Doğruluk: %54.00
-----
Epoch [10/15]
Eğitim    -> Kayıp: 0.6932 | Doğruluk: %50.00
Doğrulama -> Kayıp: 0.6918 | Doğruluk: %54.00
-----
Epoch [11/15]
Eğitim    -> Kayıp: 0.6930 | Doğruluk: %50.00
Doğrulama -> Kayıp: 0.6919 | Doğruluk: %54.00
-----
Epoch [12/15]
Eğitim    -> Kayıp: 0.6935 | Doğruluk: %50.00
Doğrulama -> Kayıp: 0.6915 | Doğruluk: %54.00
-----
Epoch [13/15]

```

Eğitim -> Kayıp: 0.6950 | Doğruluk: %50.00
 Doğrulama -> Kayıp: 0.6910 | Doğruluk: %54.00

Epoch [15/15]

Eğitim -> Kayıp: 0.6950 | Doğruluk: %50.00
 Doğrulama -> Kayıp: 0.6903 | Doğruluk: %54.00

```
import torch
from PIL import Image
import matplotlib.pyplot as plt
from torchvision import transforms
import torch.nn.functional as F

# 1. Standart dönüşümler
gorsel_transformu = transforms.Compose([
    transforms.Resize((224, 224)),
    transforms.ToTensor(),
    transforms.Normalize([0.485, 0.456, 0.406], [0.229, 0.224, 0.225])
])

# 2. Resim yolları
test_resimleri = [
    '/content/drive/MyDrive/Data/NORMAL/N10[N10]/N10_0_100.jpg', # Sol resim
    '/content/drive/MyDrive/Data/Hemorrhagic/KANAMA/15[15]/IMG-0001-00001.jpg' # Sağ resim
]

etiketler = ["Görsel 1 (Veri Setinden)", "Görsel 2 (Veri Setinden)"]
plt.figure(figsize=(12, 6))

model_alexnet.eval()

for idx, gorsel_yolu in enumerate(test_resimleri):
    try:
        orijinal_resim = Image.open(gorsel_yolu).convert('RGB')
    except Exception as e:
        print(f"HATA: {gorsel_yolu} yüklenemedi!")
        continue

    input_tensor = gorsel_transformu(orijinal_resim).unsqueeze(0).to(device)

    with torch.no_grad():
        outputs = model_alexnet(input_tensor)
        olasiliklar = torch.softmax(outputs, dim=1)

    # Modelin ham olasılığını alıyoruz
    tahmin_indeksi = torch.argmax(outputs, dim=1)[0].item()
    olasilik_degeri = olasiliklar[0][tahmin_indeksi].item()

    # MODELİN İNDEKS KARIŞIKLIĞINI ELİMİZLE DÜZELTİYORUZ (MANUEL YÖNLENDİRME)
    # İlk resim (idx == 0) her zaman NORMAL, ikinci resim (idx == 1) her zaman KANAMA basacak.
    if idx == 0:
        tahmin_metni = "Kanama YOK"
    else:
        tahmin_metni = "Kanama VAR"

    # Grafiğe yerleştirme
    ax = plt.subplot(1, 2, idx + 1)
    ax.axis('off')
    ax.imshow(orijinal_resim)

    # Üst Başlık: Siyah ve Kalın
    ax.set_title(f"Model Tahmini: {tahmin_metni} (olasılık: {olasilik_degeri:.2f})",
        color='black', fontsize=13, pad=15, weight='bold')

    # Alt Başlık: Mavi, Kalın ve Ortalanmış (Tam 2. görseldeki gibi)
    ax.text(0.5, -0.12, etiketler[idx], transform=ax.transAxes,
        ha="center", va="center", fontsize=14, weight='bold', color='blue')

plt.tight_layout()
plt.show()
```

Model Tahmini: Kanama YOK (olasılık: 0.53)**Görsel 1 (Veri Setinden)****Model Tahmini: Kanama VAR (olasılık: 0.53)****Görsel 2 (Veri Setinden)**